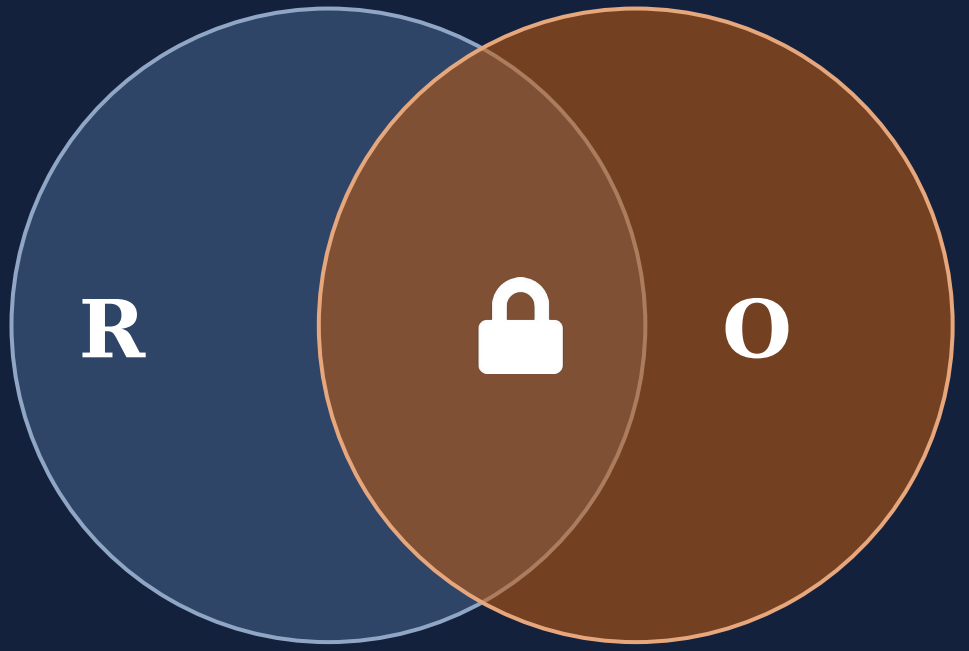
Regression across a register and a cohort, without pooling

Privacy-preserving vertical federated linear regression: motivation, protocol and results



October 2026

What we will cover

1

The problem

Why join a research cohort to an agency register, and what must stay private

2

Building blocks

Differential privacy, homomorphic encryption, private set intersection

3

The protocol

Six phases, from setup to confidence intervals

4

Guarantees

What each party learns, and what is proven

5

Choosing parameters

Two settings; from a budget to noise, ridge and sample size

6

Results

Simulation, real data, encrypted runs and other baselines

1

The problem

Joint regression across data that cannot be pooled

Two datasets, one question, no pooling



Researcher R

- Smaller cohort with rich domain variables: health, education, surveys
- Wants to know whether these predict an administrative outcome
- Holds features X_R and identifiers



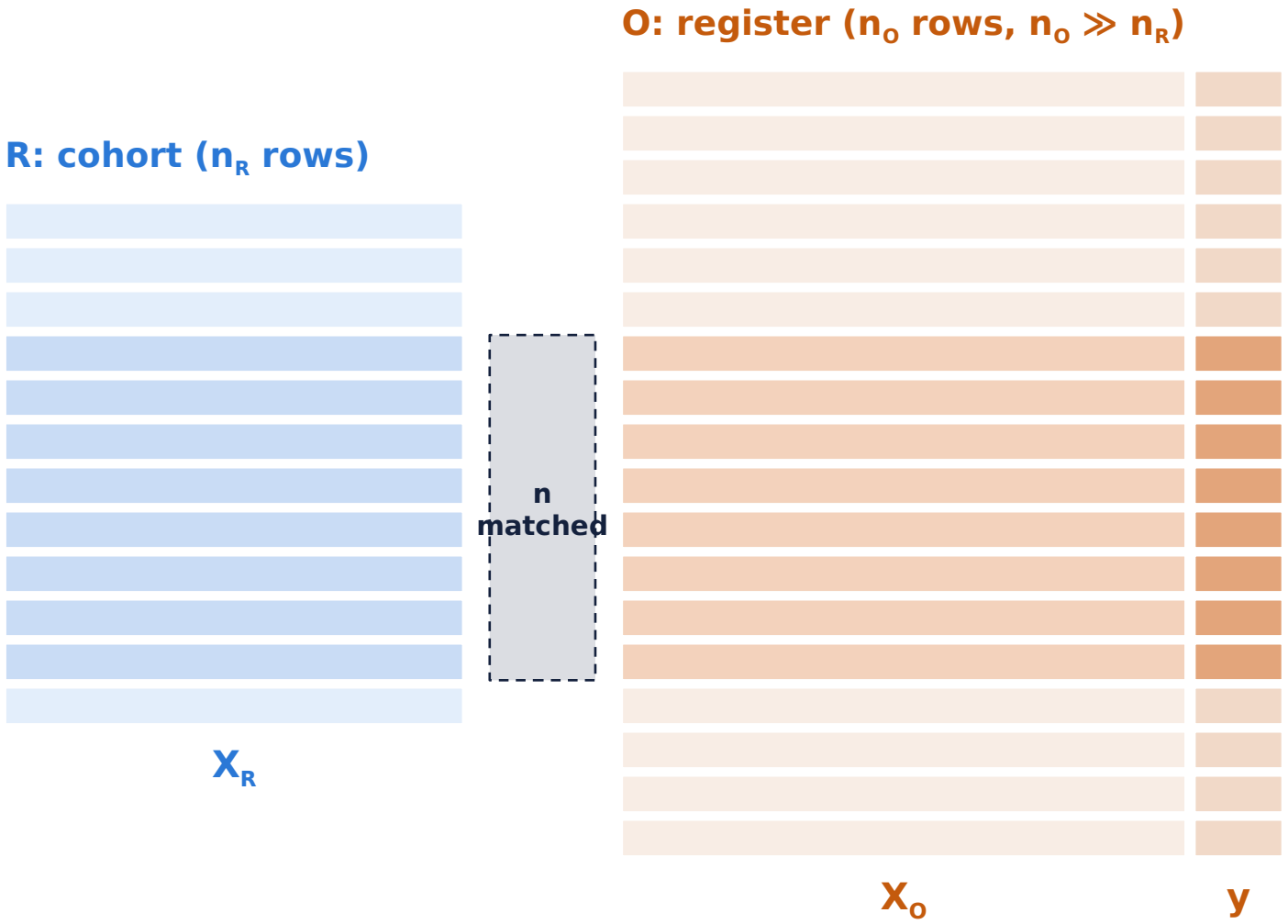
Organization O (e.g. StatCan)

- Population register: tax filings, income, benefits
- Holds the outcome y and features X_O
- Cannot release records; law and governance bar direct sharing

6–8 weeks of vetting and approvals today before a researcher can link a cohort to agency records in a Research Data Centre. Neither dataset alone answers the question.

Same people, different variables

Vertically partitioned data: overlapping rows, disjoint columns



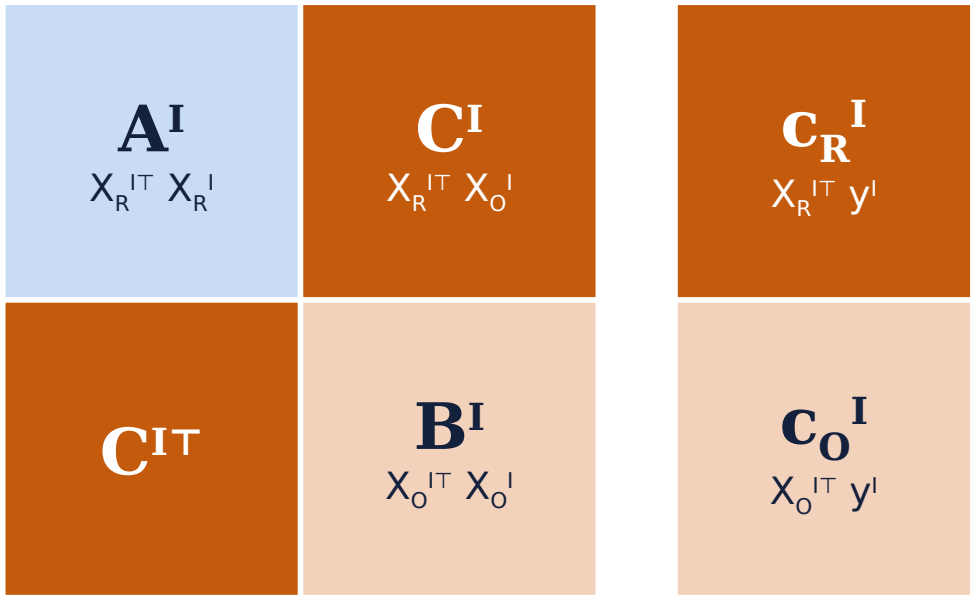
Goal

- Fit y on [X_R | X_O] over the n matched people
- Neither party sees the other's records
- O must not learn who is in R's cohort
- The released model must not expose any individual

What has to be computed

Least squares needs only a few sums over the matched rows, and those sums span both parties

$$\hat{\beta} = (Z^\top Z)^{-1} Z^\top y^I, \qquad Z = \left[X_R^I \mid X_O^I \right]$$



$G^I = Z^\top Z$ $\qquad$ $c^I = Z^\top y^I$

R's rows in I only: R computes it exactly, and it never leaves R

O's rows π(I) only, as is $(y^\top y)^I$: depends on which rows match, not on how they pair

Cross-party: pairs R's row i with O's row π(i), so it depends on the bijection π

I = the cohort members found in O's register. **π** = the bijection from I to O's matching rows; X_O^I and y^I list O's rows π(i) in the order of I. Everything that touches O's data is computed under encryption and released to R only with noise. From here on we drop the superscript I.

Three requirements

Input privacy

O learns nothing about R's features. Its whole view can be simulated from public sizes.

Cohort privacy

O learns nothing about who is in R's cohort, not even its exact size.

Output privacy

Everything R receives is differentially private for each person's O-side data (x_O, y).

Honest-but-curious parties. Both follow the protocol but analyse everything they receive. R and O each hold a share of the decryption key. By design, R learns which members of its cohort are in the register.

Where this work sits

Approach	Where the data sit	Record linkage	Output privacy	Gap for this setting
Central DP regression (e.g. AdaSSP)	One trusted curator holds the joined table	Not needed	Differential privacy	Assumes the pooled data exist
Federated learning with encryption	Split across parties	Assumed, or a separate step	Usually none	The model can leak individuals
Private set intersection	Split across parties	Yes	Not applicable	Linkage only, no model
This protocol	Split, never pooled	Labeled PSI: O learns nothing	DP on all R receives; budget enforced by O	Protects O's side; R learns which of its cohort match

Each ingredient has a precedent: DP noise at collective decryption (UnLynx, Drynx, SPINDLE, FAMHE; Kluczniak–Santato for the cryptographic half) and noisy sufficient statistics for vertical regression (Xu–Yuan–Wu; Krichene et al.). **What is new is the combination:** one decryption share whose noise is both the zCDP noise and the CKKS flooding, R’s block exact, a ridge independent of the estimation noise, and a ledger for adaptive runs.

2

Building blocks

Differential privacy, homomorphic encryption, private set intersection

Differential privacy in one slide

A release is private if no single person's data changes it by much

$$\Pr[\mathcal{M}(D) \in S] \;\leq\; e^{\varepsilon}\,\Pr[\mathcal{M}(D') \in S] + \delta$$

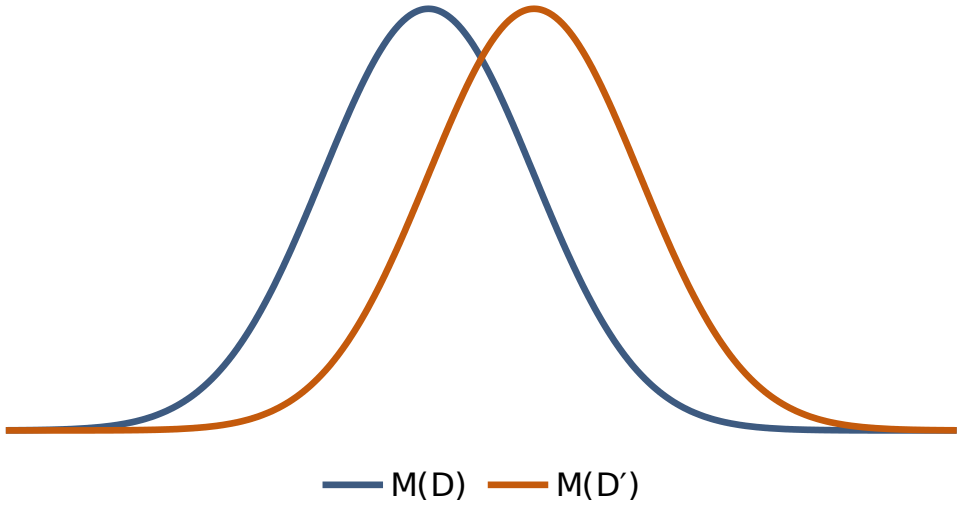
- D and D′ differ in one person's O-side data (x_o, y): replace-one adjacency
- ε caps how much one person can shift the odds of any outcome (a factor e^{ε})
- δ is a small slack; take δ ≤ 1/(10N) for a register of N people
- Post-processing is free: anything computed from a DP release stays DP

The Gaussian mechanism

$$\mathcal{M}(D) = f(D) + \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \Delta \cdot s(\varepsilon, \delta)$$

$$\Delta = \max_{D \sim D'} \|f(D) - f(D')\|_2$$

Outputs on neighbouring datasets



The two curves are shifted by the sensitivity Δ. The wider the noise relative to Δ, the harder it is to tell them apart.

Δ_s: how far one person can move a release

DP protects the people in O's register from R, the only party that sees decrypted output

Who is protected

Neighbours D ~ D′: one matched person's (x_O, y) is replaced by any other value within the bounds. Their R row $\dot{x}$ and the match set I stay fixed.

- R already knows $\dot{x}$ and I (the linkage reveals I by design: outside DP).
- O sees only ciphertexts (encryption, not DP).

What release s contains

s	Stage	Entries released
1	stage 1	triu B, C
2	stage 2	triu B, C, c_R, c_O, y^Ty

A = $\dot{X}^T\dot{X}$ is never released: R computes it itself, and it is the same in both neighbours.

The sensitivity Δ_s

$$\Delta_s = \sup_{D \sim D'} \left\| z_s(D) - z_s(D') \right\|_2$$

With $\dot{x}$ fixed and $x = x_O$, a replacement $(x, y) \to (x', y')$ moves:

$$\begin{aligned} \operatorname{triu} B: &\quad \operatorname{triu}\left(xx^\top - x'x'^\top\right) \\ C: &\quad \dot{x}\,(x - x')^\top \\ c_R: &\quad \dot{x}\,(y - y') \\ c_O: &\quad x\,y - x'\,y' \\ y^\top y: &\quad y^2 - y'^2 \end{aligned}$$

Every entry gets the same noise, so the squared changes add up over the blocks of a release. Stage 1 has only the first two blocks, so $\Delta_G < \Delta$. Only the upper triangle of the symmetric B is released, so each off-diagonal change counts once.

Maximizing over the bounds, with $(r, o, w) = (B_R^2, B_O^2, B_y^2)$:

$$\Delta_2^2 = \Delta^2 = \tfrac{1}{2}\,(2o + r + w)^2 + 2rw \qquad (|r - w| \le 2o)$$
$$\Delta_1^2 = \Delta_G^2 = \tfrac{1}{2}\,(2o + r)^2 \qquad (r \le 2o)$$

Clipping is what makes Δ finite, and Δ does not depend on n: more records make the signal bigger, not the noise. Upper triangle of B triu B is upper triangle of B (including diagonal). Note that B is symmetric.

How much one person can move the statistics

The sensitivity Δ sets the noise; it follows from public clipping bounds

Every row is clipped to public bounds: $\|x_R\| \le B_R$ (with the intercept), $\|x_O\| \le B_O$ and $|y| \le B_y$. Replacing one person's (x_O, y) changes B, C, c_R, c_O and y^Ty by at most Δ:

Bounds (B_R, B_O, B_y)	Δ	Comment
(1, 1, 1)	3.16	unit bounds
(0.71, 0.71, 1)	2.03	one unit row bound split between the parties
(5.59, 6.24, 2.89)	86.2	simulation population (standardized income, age, counts)

Bounds are chosen from public reference data before any data are seen; values beyond them are clipped.

- $A = X_R^T X_R$ is computed exactly by R and never released, so it adds nothing to Δ
- y^Ty is free when $B_y^2 \le 2(B_R^2 + B_O^2)$; otherwise it gets its own budget
- Δ grows with the squared bounds: tight bounds matter most

σ_s and ρ_s: one equation ties them to Δ_s

Δ_s is fixed by the bounds and ρ_s by the budget, so the noise σ_s follows

$$\rho_s = \frac{\Delta_s^2}{2\,\sigma_s^2} \qquad \Longleftrightarrow \qquad \sigma_s = \frac{\Delta_s}{\sqrt{2\rho_s}}$$

Δ_s sensitivity: how far one person can move release s
ρ_s zCDP cost: the share of the budget release s spends
σ_s noise s.d. added to every entry of release s

Why the equation holds

R sees $\hat{z}_s = z_s + N(0, \sigma_s^2\, I)$: independent noise with the same s.d. on every entry. For neighbours D ~ D′ and any order a > 1, the Rényi divergence is

$$D_a\big(\mathcal{N}(z_s(D), \sigma_s^2 I) \,\|\, \mathcal{N}(z_s(D'), \sigma_s^2 I)\big) = \frac{a\,\|z_s(D) - z_s(D')\|_2^2}{2\sigma_s^2} \le a\,\frac{\Delta_s^2}{2\sigma_s^2}$$

ρ-zCDP means this is at most aρ for every a > 1, so release s costs exactly $\rho_s = \Delta_s^2/(2\sigma_s^2)$. Costs add: $\rho_{run} = \rho_1 + \rho_2$. They add across runs too, and O's ledger caps the total.

From the run's (ε, δ) to the noise

1 Convert (ε, δ) to a zCDP budget

$$\sqrt{\rho_{\text{run}}} \gtrsim \sqrt{L + \varepsilon} - \sqrt{L}, \qquad L = \ln(1/\delta)$$

Tight conversion: $\rho_{run} = 0.01497$ at ε = 1, $\delta = 10^{-9}$.

2 Split it between the two stages

$$\rho_1 = f_1\,\rho_{\text{run}}, \qquad \rho_2 = (1 - f_1)\,\rho_{\text{run}}, \qquad f_1 = 0.25 \text{ (I and P)}$$

3 Set the noise

$$\sigma_1 = \Delta_G/\sqrt{2 f_1 \rho_{\text{run}}}$$
$$\sigma_2 = \Delta/\sqrt{2(1 - f_1)\rho_{\text{run}}}$$

I and P: $\sigma_1 \approx 1.55\sigma_2$ here. The 25% share raises σ_2 by 15% (previous: 10%, +5%).

Toy example (previous setting, $f_1 = 0.1$; ε = 1, $\delta = 10^{-9}$): $\Delta_G = 4.95$, $\sigma_1 = 90.45$; $\Delta = 6.16$, $\sigma_2 = 37.55$. Everything after the releases (λ, the gate, $\tilde{\theta}$, V_{mech}, the intervals) uses only $\hat{z}_1$, $\hat{z}_2$, A and public numbers: free post-processing.

Budgets add up, and O keeps the books

Zero-concentrated DP (zCDP) makes the accounting simple

$$\rho_s = \frac{\Delta_s^2}{2\sigma_s^2}, \qquad \rho_{\text{run}} = \rho_1 + \rho_2$$

$$\sqrt{\rho_{\text{run}}} \gtrsim \sqrt{L+\varepsilon} - \sqrt{L}, \qquad L = \ln(1/\delta)$$

- Each release s costs ρ_s; the costs of a run add up
- The run's total converts to (ε, δ); the protocol uses the tight conversion, and the formula above is a close, simple bound
- Doubling ε roughly halves the noise σ

One run = two releases



Stage 1: B and C only, 25% of ρ in settings I and P (previous: 10%), used to choose the ridge

Ledger at O

One ledger per base register, shared by all researchers. A run's full cost is charged atomically at admission, never refunded. Adaptive runs pass a privacy filter; smudging is accounted in hockey-stick divergence (share ξ_L). No decryption without O's share.

Where the noise actually lives

O adds integer noise η to the ciphertext's coefficients; R decodes it as Gaussian noise on each entry

1 Statistic in the slots

Release s is a vector z_s with one entry per slot. One person moves it by at most Δ_s in $\ell 2$.

encode → ×Ω, τ⁻¹

2 Coefficients: O adds η

$$\eta \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma_{c,s}^2)^N$$

$$\Delta_{\mathrm{enc},s} = \sqrt{2/N}\,\Omega\,\Delta_s + \sqrt{N}$$

$$\rho_s = \Delta_{\mathrm{enc},s}^2 \,/\, (2\sigma_{c,s}^2)$$

exact for the discrete Gaussian

decode → ÷Ω, τ

3 What R decodes

$$\sigma_s = \sigma_{c,s}\sqrt{N/2}\,/\,\Omega$$

Noise with s.d. σ_s on every released entry: the picture of the previous slide.

Why the slot picture is enough

$$\sigma_s = \frac{\Delta_s + N/(\sqrt{2}\,\Omega)}{\sqrt{2\rho_s}} \approx \frac{\Delta_s}{\sqrt{2\rho_s}}$$

With $N = 2^{14}$ and $\Omega_f = 2^{61.8}$–$2^{71.9}$ the extra term is below 10^{-10}. Exact integer sampling of the discrete Gaussian makes the guarantee exact.

η also hides the CKKS error

$$\mathrm{SD}(\eta + e,\ \eta) \le \frac{\|e\|_2}{\sqrt{2\pi}\,\sigma_c} \le 2^{-\kappa} \quad \text{if} \quad \sigma_c \ge \frac{2^{\kappa} B_{\mathrm{tot}}}{\sqrt{2\pi}}$$

e depends on the data and O's key share, so it is hidden (κ = 40–46, from ε, δ). As $\sigma_{c,s} = \sigma_s \cdot \Omega_f \cdot \sqrt{(2/N)}$, O raises Ω_f until:

$$\log_2 \Omega_f \approx \underbrace{\kappa}_{40} + \underbrace{\log_2 B_{\mathrm{tot}}}_{26.6} + \underbrace{\log_2\sqrt{N/2}}_{6.5} - \underbrace{\log_2\sqrt{2\pi}}_{1.3} - \underbrace{\log_2 \sigma_2}_{9.2} \approx 62.6$$

Price: $\delta_{DP} = \delta_{run} - (1 + e^{\varepsilon}) \cdot n_{rel}(\delta_B + 2^{-\kappa} + \ldots)$.

How the σ's drive the statistics: σ_1 sets the radius $r_1 = q \cdot \sigma_1$ (exact quantile: q = 7.42 in I, 4.44 in P, at p = 11) and so λ; σ_2 sets the ridge target $2\vartheta^* \sigma_2 \sqrt{p}$, the gate and $V_{mech} \propto \sigma_2^2$. Records needed for a given coefficient accuracy grow like $\sigma_2^2 \propto \Delta^2/\rho$.

Threshold CKKS: one key, split between R and O

Nobody ever holds $s = s_R + s_O$, so neither party can decrypt on its own

CKKS in one box

A ciphertext is a pair (c_0, c_1) of polynomials mod q. It packs a vector of N/2 = 8,192 real numbers ($N = 2^{14}$), each scaled by Ω and rounded.

Decryption with the secret key s:

$$c_0 + c_1 s \;=\; \mathrm{Ecd}(z) + e \pmod q$$

What O can do with R's ciphertexts, holding no key:

- add them
- multiply them by its own plaintext columns (x_O, y)
- rotate slots, to sum a vector into one slot (EvalSum)

Depth 2 and no ciphertext × ciphertext products, so no relinearization key. Security rests on Ring-LWE: without s, c_0 looks uniformly random.

Joint key generation (setup)

From a common random polynomial a, each party draws its own secret share and error, and publishes one polynomial:

R : $p_{0,R} = -a\, s_R + e_R$

O : $p_{0,O} = -a\, s_O + e_O$

$$pk = \big(p_{0,R} + p_{0,O},\ a\big) = \big(-a\, s + e,\ a\big)$$

Rotation keys for slot shifts 2^i are made the same way. Each share stays with its owner.

Upload under the joint key, at half the size

R : $c_0 = -a\, s_R + e + \mathrm{Ecd}(v), \qquad a = \mathrm{PRG}(\mathrm{seed})$

O : $c_0 \leftarrow c_0 - a\, s_O \quad \Rightarrow \quad c_0 + a\, s = \mathrm{Ecd}(v) + e$

R sends c_0 and the short seed of a, not a itself. O's subtraction turns (c_0, a) into a ciphertext under s.

This is standard multiparty CKKS (Mouchet et al., 2021) with two parties and additive key shares. What is specific to us is how the decryption is used: next slide.

Threshold decryption, with the DP noise built in

O adds its key share and the noise in one step; R finishes the decryption and sees only noisy statistics

Decryption needs both shares: $s = s_R + s_O$: $c_0 + c_1 s = \underbrace{(c_0 + c_1 s_O)}_{\text{needs O}} + \underbrace{c_1 s_R}_{\text{needs R}}$

Our release (phase 4)

O : $c_0' = c_0 + p_0 u + e_0, \quad c_1' = c_1 + au + e_1$

$h_O = c_0' + c_1' s_O + \eta, \quad \eta \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma_c^2)^N$

send (c_1', h_O)

R : $v = h_O + c_1' s_R = \mathrm{Ecd}(z) + e + \eta$

1 Fresh u, e_0, e_1 every release

The evaluated c_1 depends on O's plaintext columns; adding $au + e_1$ makes c_1' a fresh RLWE sample that looks uniform.

2 Add η inside O's share

η is the DP noise and also drowns the CKKS error e, up to $2^{-\kappa}$ ($\kappa = 40$–46). It is fresh too, so two releases cannot be differenced.

3 Never send c_0'

The proof needs it: without c_0', u and e_1 appear only in c_1', which then looks uniform. With it, R would get $c_1' s_O + \eta$: O's key share masked only by η.

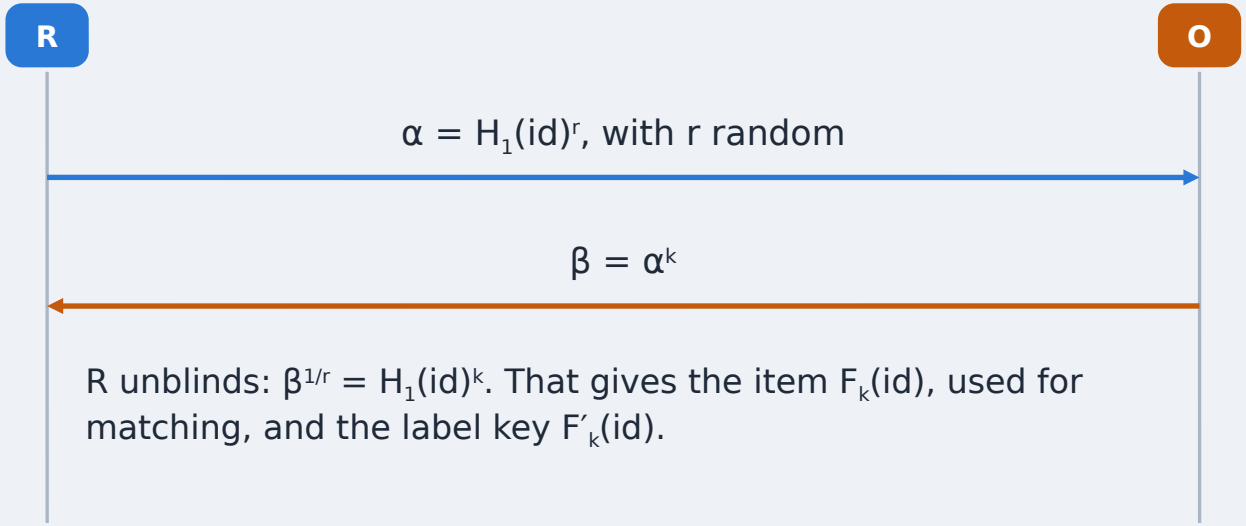
O's view reveals nothing. It never receives R's decryption share. Scope: R's decrypted v_s must never leave R, and anything R discloses later is noiseless to O.

R learns the noisy statistics. Plus a c_1' that looks random, so its view can be simulated from the noisy statistics alone: the DP guarantee (if R samples its errors honestly).

The OPRF: a keyed hash that needs O

R learns $F_k(id)$ for its own ids; O never sees the ids, and R never sees the key k

$$F_k(x) = H_2\left(x,\ H_1(x)^k\right)$$ 2HashDH (RFC 9497); O's key k is fresh each run



Why it is safe

- O sees α, a uniformly random group element: nothing about the id.
- R gets F_k only for ids it sends. Without k it cannot compute F_k for any other id.
- O answers exactly n_R^{max} queries per run (R pads with dummies) and logs them against a quota.
- A fresh k every run, so F_k values from different runs cannot be linked.

Why we need it: identifiers such as social insurance numbers are a small, guessable space, so a plain hash could be inverted by trying every value. With the OPRF every evaluation needs O, so R cannot test ids offline, and the PSI messages carry no structure. Toy example: R holds 104, 221, 317, pads with dummy 9d1 to $n_R^{max} = 4$, and makes four calls.

Labeled PSI: who matches, and where

R learns its match set I and positions π(i) = τ(j) in O's shuffled register; O learns only the padded sizes

1 O prepares

Pad the register to $\bar{n}_O$ with dummies and draw a fresh shuffle τ. Row j becomes the item $F_k(\mathrm{id}_j)$, labeled with its encrypted position. Hash the items into bins; per bin b:

$$P_b(X) = \prod_{j \in b} \left(X - F_k(\mathrm{id}_j)\right)$$
$$L_b\left(F_k(\mathrm{id}_j)\right) = \mathrm{Enc}_{F'_k(\mathrm{id}_j)}\left(\tau(j)\right)$$

2 R asks, O evaluates

R cuckoo-hashes its items, at most one per bin, encrypts them under its own BFV key, and sends their powers q, q^2, q^4, … to keep the depth low. O evaluates every P_b and L_b under encryption and replies:

$$\mathrm{Enc}_R\left(P_b(q)\right), \quad \mathrm{Enc}_R\left(L_b(q)\right)$$

3 R decrypts

$$P_b(q) = 0 \;\Rightarrow\; \text{match}$$
$$\pi(i) = \mathrm{Dec}_{F'_k(\mathrm{id}_i)}\left(L_b(q)\right) = \tau(j)$$

No match: $P_b(q) \neq 0$ and the label value means nothing. The reply reveals nothing else about O's register.

Toy example

O's padded row j	150	7e5	221	088	399	104
position τ(j)	1	2	3	4	5	6

7e5 is O's dummy row. R queries 104, 221, 317 and its dummy 9d1: 104 matches with label 6 and 221 with label 3. So I = {104, 221}, π(104) = 6 and π(221) = 3.

What each party learns

O: only $n_R^{\max}$ and $\bar{n}_O$, both public.

R: I and the positions, which are uniformly random because τ is fresh. R needs both: I for its block A, the positions to line up its upload.

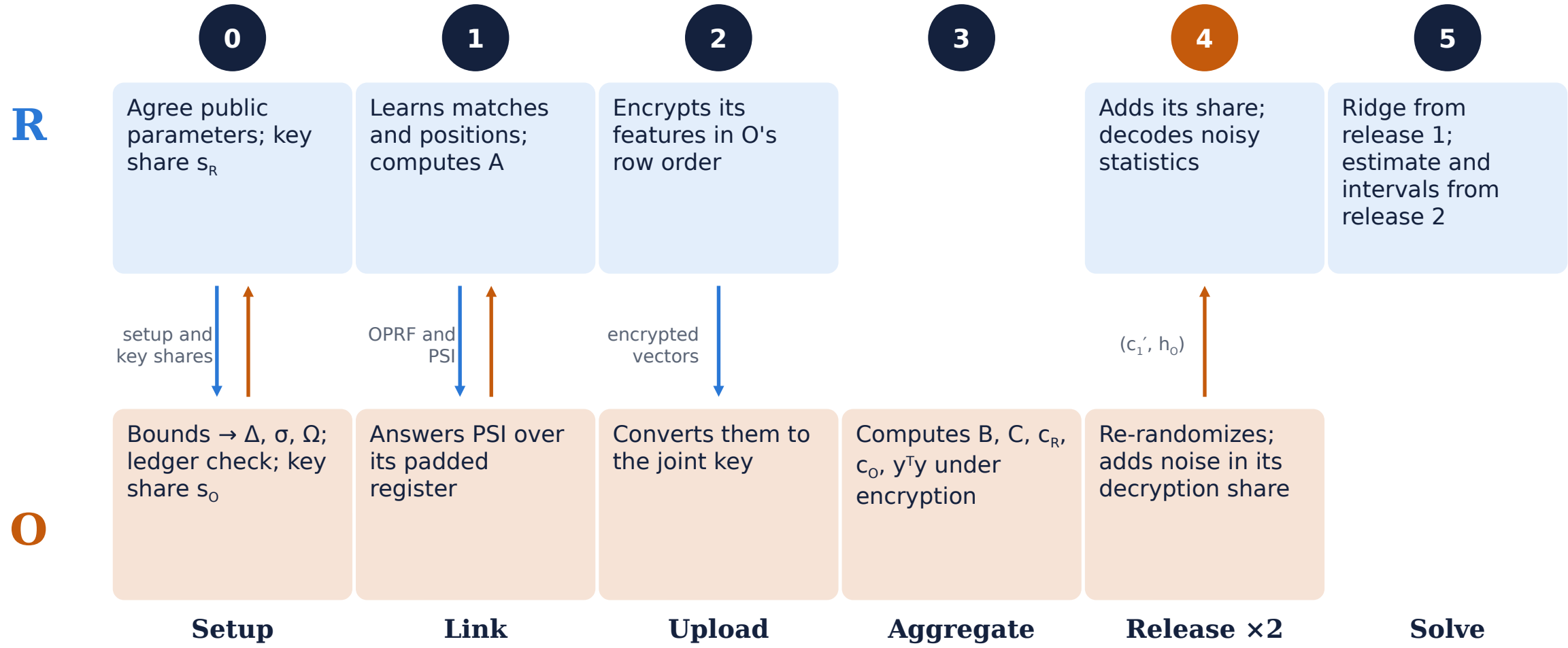
3

The protocol

Six phases, from setup to confidence intervals

The protocol at a glance

What each party does, and what crosses the wire



O never receives a decryption share from R; R never receives the part of a ciphertext that would expose O's key share.

Phase 0: setup

Everything that sets the noise is public and fixed before any data are touched

R + O Agreed in public

- Study scope Φ, on public attributes only (province, birth-year band)
- Caps: cohort n_R^{max}; register padded to $\bar{n}_O$ with dummy rows
- Clipping bounds B_R (with the intercept), B_O, B_y, from public reference data
- Budget (ε, δ) and the statistical setting: I ($f_1 = 0.25$, $\vartheta^* = 2.5$, exact r_1, intervals) or P ($f_1 = 0.25$, $\vartheta^* = 1.5$, $\alpha_L = 0.75$, no intervals)

O Derived by O

- Sensitivities Δ and Δ_G from the bounds
- Flooding κ, noise σ_1, σ_2 and scale Ω_f (Calibrate)
- Ledger: admit and charge the full cost, or refuse
- A fresh secret shuffle τ of its rows and a fresh OPRF key

R + O Split key generation

Joint public key, secret $s = s_R + s_O$, and rotation keys only. Neither party can decrypt alone.

Phase 1: link

Labeled PSI tells R which cohort members are in the register, and where

R Pad the cohort identifiers with random dummies to exactly n_R^{max}

R + O OPRF: R gets $F_k(id)$ for each padded id; O evaluates exactly n_R^{max} calls

R Cuckoo-hash the hashed ids; send encrypted queries under R's own key

O Evaluate membership and label polynomials over its padded register; reply

R Decrypt: match set I, and positions $\pi(i) = \tau(j)$ in O's shuffled order

R Compute its own block $A = \Sigma_{i \in I} \dot{x}_i \dot{x}_i^T$ exactly, locally

After phase 1

- **R knows I and the positions τ(j) of its matches**
- R has A exactly; it never leaves R
- **O knows only n_R^{max} and $\bar{n}_O$**
- The positions are uniformly random, because τ is fresh every run

Phase 2: upload

R lines its data up with O's rows, encrypts them, and sends them over

R

Vectors over all $\bar{n}_o$ positions

- Selection b: 1 at a matched position, 0 elsewhere
- Features $\dot{x}_j$: R's clipped feature j (and the intercept) at matched positions, 0 elsewhere
- In O's shuffled order, so O can multiply them by its own columns position by position

Why this hides everything from O

O sees only ciphertexts, which look random without R's key share. Matched and unmatched positions are indistinguishable, so O cannot tell who is in the cohort.

R : $c_0 = -a\, s_R + e + \mathrm{Ecd}(v), \quad a = \mathrm{PRG}(\mathrm{seed})$ O : $c_0 \leftarrow c_0 - a\, s_O$

R encrypts with its own share and a short seed (half the upload); O folds in its share to convert to the joint key.

Phase 3: aggregate

O multiplies R's encrypted vectors by its own plaintext columns and sums

$$\mathrm{Enc}(B_{jk}) = \sum_i \mathrm{Enc}(b_i)\, X_{O,ij}\, X_{O,ik}$$

$$\mathrm{Enc}(C_{jk}) = \sum_i \mathrm{Enc}(\dot{X}_{R,ij})\, X_{O,ik}$$

$$\mathrm{Enc}(c_{O,k}) = \sum_i \mathrm{Enc}(b_i)\, X_{O,ik}\, y_i$$

$$\mathrm{Enc}(c_{R,j}) = \sum_i \mathrm{Enc}(\dot{X}_{R,ij})\, y_i$$

b: selection vector; $\dot{x}_R$: R's features in O's order (zero off the match set)

Sum chunks, then EvalSum

A register longer than one ciphertext is split into chunks; slot sums use rotations.

Pack: one entry per slot

Every released number sits alone in its own slot of a packed ciphertext.

Two packed ciphertexts

M_1 holds only B and C (for stage 1); M_2 holds every entry (stage 2).

Unmatched positions contribute zero, because b and $\dot{x}_R$ are zero there. The result is the matched-row statistics, still encrypted. Multiplicative depth is 2.

Phase 4: the release

O adds the privacy noise inside its own share of the decryption

$$
\begin{aligned}
\mathrm{O}: \quad & c_0' = c_0 + p_0 u + e_0, \qquad c_1' = c_1 + a u + e_1 \\
& h_O = c_0' + c_1' s_O + \eta, \qquad \eta \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma_c^2)^N \\
& \text{send } (c_1', h_O) \\
\mathrm{R}: \quad & v = h_O + c_1' s_R = \mathrm{Ecd}(z) + e + \eta
\end{aligned}
$$

c_0' never leaves O

Needed by the proof: c_1' then looks uniform. With it, R would get $c_1' s_O + \eta$.

η does two jobs

The DP noise on the statistics; it also floods every encryption error (to $2^{-\kappa}$, κ = 40–46).

Fresh every time

New u, e_0, e_1 and η for each release; the ledger was charged at admission.

What R ends up with: the statistics plus discrete-Gaussian noise of the planned size, and a c_1' that looks random. R's view can be simulated from the noisy statistics alone, which is exactly the DP guarantee.

Two releases: pick the ridge, then estimate

The ridge must not depend on the noise in the release it is applied to

Stage 1 (25%)

Release of B and C only, noise σ_1

→

$\hat{G}_1$ = A ⊕ noisy B, C

R adds its exact block A; finds a lower bound on $\lambda_{min}(G)$

→

Choose (λ, Ψ)

Ridge just large enough that $\vartheta \geq \vartheta^*$ (I: 2.5, P: 1.5); Ψ ridges only O's block if that suffices

→

Stage 2 (75%)

All statistics, noise σ_2; gate $\hat{\vartheta} \geq 1$

$$\lambda = \max\{0,\ 2\vartheta^\star \sigma_2 \sqrt{p} + r_1 - \lambda_{\min}(\hat{G}_1)\}, \qquad r_1 = \sigma_1\, q_{1-\alpha_L}$$

$$\tilde{\theta} = (\hat{G}_2 + \lambda\Psi)^{-1}\hat{c}_2, \qquad \hat{\vartheta} = \frac{\lambda_{\min}(\hat{G}_2 + \lambda\Psi)}{2\sigma_2\sqrt{p}} \ \geq\ 1$$

Because (λ, Ψ) comes from stage 1 and A only, it is independent of the stage-2 noise, which keeps the error analysis and the intervals valid. ϑ measures how far the noisy Gram matrix is from singular. q is an exact Monte Carlo quantile of the stage-1 noise: 7.42 (I) or 4.44 (P) at p = 11; the previous setting's union bound gave 11.53.

Phase 5: estimates and intervals

Everything here is post-processing at R: it costs no extra privacy budget

Two targets

$$\theta_\lambda = (G + \lambda\Psi)^{-1}c \qquad \text{and} \qquad \beta:\ \mathrm{E}[\tilde{y} \mid \tilde{X}] = \tilde{X}\beta \quad (\hat{\beta} = G^{-1}c)$$

θ_λ is what the protocol estimates directly (the ridge target on the clipped data); β is the coefficient of the working linear model on the clipped data.

Parametric bootstrap of the noise

- R knows the noise distribution exactly, so it simulates many fresh noise draws around its release
- This gives the mechanism's covariance V_{mech}, and corrects the residual sum of squares for the noise
- Add the usual sampling variance to get the total V

Interval for θ_λ (Wald)

$$\tilde{\theta}_j \ \pm\ z_{1-\alpha/2}\sqrt{(V_{\mathrm{mech}})_{jj}}$$

Covers the protocol's own target at the nominal level. Setting I only: setting P reports no intervals.

Interval for β (bias-aware)

$$\tilde{\theta}_j \ \pm\ \mathrm{cv}_\alpha\big(\bar{b}_j/\sqrt{V_{jj}}\big)\sqrt{V_{jj}}, \qquad \bar{b}_j = \lambda\,\|\Psi\tilde{P}e_j\|\,\hat{B}_\beta$$

Widened for the worst-case ridge bias (Armstrong, Kolesár and Kwon); needs a bound on ‖β‖.

How the Wald interval is computed

V_{mech}, the covariance due to the DP noise, by parametric bootstrap at R: free post-processing

1 Simulate the noise

For b = 1, ..., B, around R's own release:

$$\mathbf{E}_b^* = \mathbf{E}_b^{*\top}, \quad E_{b,ij}^* \sim \mathcal{N}(0, \sigma_2^2) \text{ on } B, C;\ 0 \text{ on } A$$

$$\mathbf{f}_b^* \sim \mathcal{N}(0,\ \sigma_2^2 I_p)$$

$$\theta_b^* = \left(\hat{G}_2 + \mathbf{E}_b^* + \lambda\Psi\right)^{-1}\left(\hat{c}_2 + \mathbf{f}_b^*\right)$$

$$V_{\text{mech}} = \frac{1}{B-1}\sum_{b=1}^{B}\left(\theta_b^* - \bar{\theta}^*\right)\left(\theta_b^* - \bar{\theta}^*\right)^\top$$

keep b only if $\left\|\tilde{P}^{1/2}\,\mathbf{E}_b^*\,\tilde{P}^{1/2}\right\|_{\text{op}} \le \frac{1}{2}$

Without this truncation the bootstrap mean and covariance do not exist. Use antithetic pairs (E^*_b, f^*_b) and ($-E^*_b$, $-f^*_b$).

2 What it estimates

First order in the noise:

$$\tilde{\theta} - \theta_\lambda \ \approx\ \tilde{P}\left(\mathbf{f} - \mathbf{E}\,\theta_\lambda\right), \qquad \tilde{P} = \left(\hat{G}_2 + \lambda\Psi\right)^{-1}$$

$$V_{\text{mech}} \ \approx\ \sigma_2^2\ \tilde{P}\left(I + K(\tilde{\theta})\right)\tilde{P}$$

$$K(\theta)_{ii} = \sum\nolimits_j \chi_{ij}\,\theta_j^2, \qquad K(\theta)_{ik} = \chi_{ik}\,\theta_i\theta_k \quad (i \ne k)$$

χ_{ij} = 0 if i and j are both in R's block A (never noised), else 1.

I: noise f on $\hat{c}_2$. K(θ): noise E on $\hat{G}_2$, times θ.

R can use this closed form instead of the bootstrap: same first-order guarantee, no Monte Carlo error.

3 The interval

$$\tilde{\theta}_j \ \pm\ z_{1-\alpha_{\text{CI}}/2}\sqrt{(V_{\text{mech}})_{jj}}$$

$\alpha_{CI} = \alpha - \alpha_L - \alpha_U = 0.045$, so z = 2.005 (nominal 95.5%)

Target

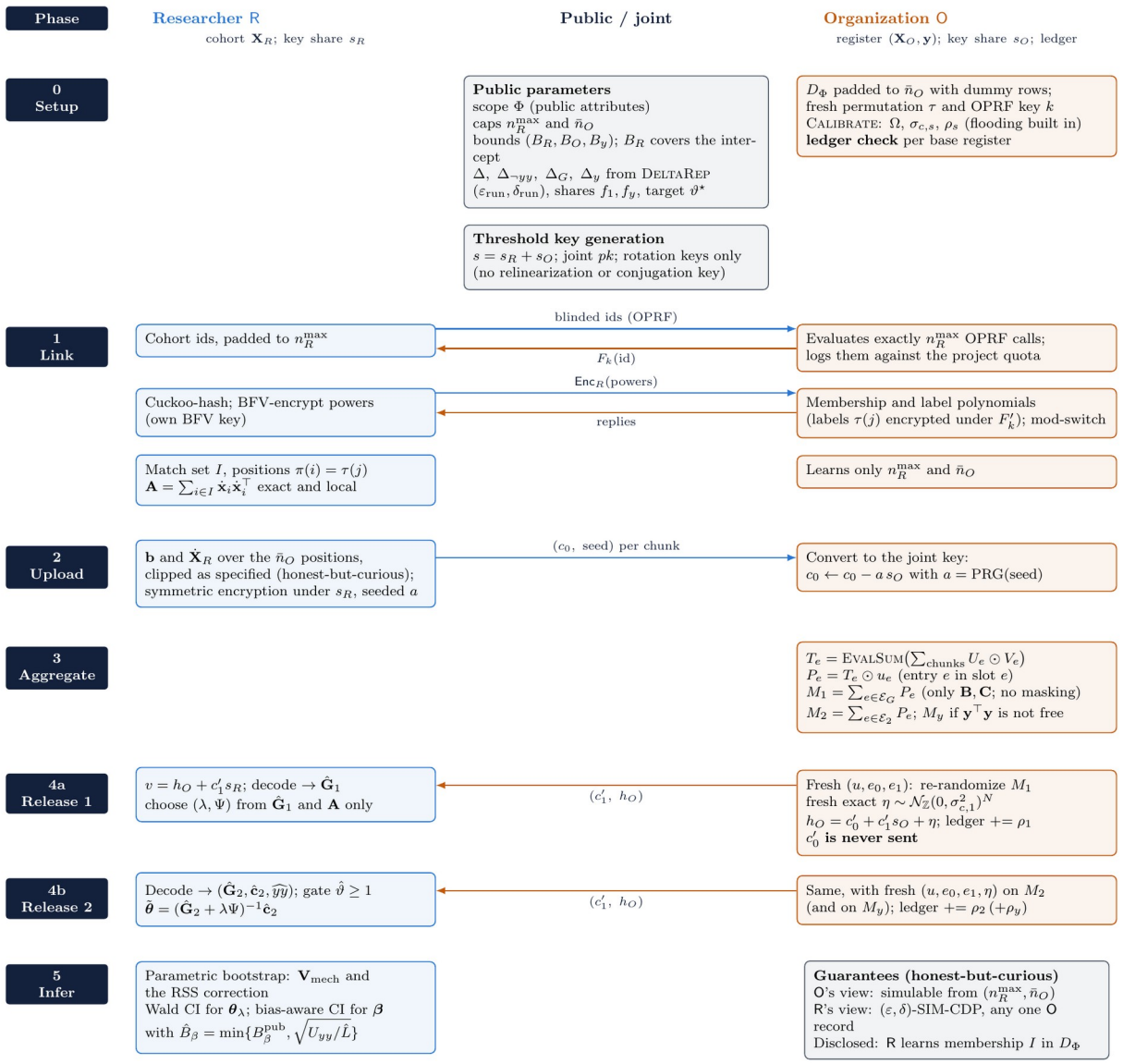
$$\theta_\lambda = (G + \lambda\Psi)^{-1}\,c$$

a fixed function of the data, so only the noise enters: no sampling variance.

First order: relative error $O(1/\vartheta_0) + O(B^{-1/2})$. Under setting I the intervals cover 0.945–0.966 in all 70 synthetic cells and 0.945–0.964 in 48 real-data cells (target 0.955). The interval for β adds sampling variance, $V = V_{mech} + \tilde{s}^2\,\tilde{P}\,\hat{G}_2\,\tilde{P}$, and is widened for the ridge bias.

The whole protocol on one page

Blue arrows: R to O. Orange arrows: O to R.



4

Guarantees

What each party learns, and what is proven

What each party learns

Against honest-but-curious parties

	Learns	Does not learn	Why
O	The public sizes n_R^{max} and $\bar{n}_O$	R's features, who is in the cohort, or which of its rows match	Ring-LWE in R’s key share; OPRF and PSI security; O never receives R’s decryption share or v_s. O’s view only: what R discloses is noiseless to O
R	Its own data; which of its cohort match, and where; the DP-noised statistics	Anything about one person’s O-side data beyond (ε, δ)-DP; nothing else about O’s register (G4)	Noise in O’s share; view simulated from the noisy statistics (SIM-CDP). G4: random-oracle model, CDH; PSI-noise layer in the textbook BFV model

Assumed

- Honest-but-curious parties; R samples its encryption errors honestly
- Ring-LWE; CDH (OPRF); private, authenticated channels
- Across runs: an honest ledger; colluding, adaptive researchers stay $(\varepsilon_L, \delta_L)$-DP

Not covered

- A deviating researcher (skips clipping, inflates errors): needs zero-knowledge proofs
- Hiding match status from R: circuit-PSI, at a higher sensitivity
- Results R publishes, as seen by O (O knows the noise)

Why the CKKS scale has to be large

Hiding the encryption error statistically costs about 40 extra bits

$$\mathrm{SD}(\eta + e,\ \eta) \ \le\ \frac{\|e\|_2}{\sqrt{2\pi}\,\sigma_c} \ \le\ 2^{-\kappa} \quad \text{if} \quad \sigma_c \ge \frac{2^{\kappa} B_{\mathrm{tot}}}{\sqrt{2\pi}}$$

$$\sigma_2 = \sigma_c \sqrt{N/2}\,/\,\Omega$$

The privacy budget fixes the noise σ_2 seen in the statistics. So the scale Ω_f must be large enough that η is also 2^{κ} times the error, and $\sigma_c \ge 2^{42}$:

$$\log_2 \Omega_f \ \approx\ \underbrace{\kappa}_{40} + \underbrace{\log_2 B_{\mathrm{tot}}}_{26.6} + \underbrace{\log_2 \sqrt{N/2}}_{6.5} - \underbrace{\log_2 \sqrt{2\pi}}_{1.3} - \underbrace{\log_2 \sigma_2}_{9.2} \ \approx\ 62.6$$

Example: setting I, ε = 1, δ = 10^{-9}, simulation bounds (σ_2 = 575); $B_{tot} = 2^{26.6}$ is proven for OpenFHE's key switching.

Fixed parameters: N = 2^{14}, chain [60, 59, 59, 59] + P [60, 60] bits, log QP = 357 < 438 (128-bit limit). κ is derived from (ε, δ); q_0 needs at most 98 of its 119 bits.

ε	κ	$\log_2 \Omega_f$
0.5	40	61.8
1	40	62.75
2	40	63.7
4	41	65.6–65.7
8	46	71.6–71.9

$2^{17.8}$–$2^{18.8}$

measured error in OpenFHE 1.5.1, 7.8–8.7 bits below the proven worst case $B_{tot} = 2^{26.6}$. O calibrates from the bound, never from the observed error; OpenFHE's noise estimation is not used.

5

Choosing parameters

From a privacy budget to noise, ridge and the sample size needed

Two settings of the same protocol

Setting I (inference)

- Exact radius $r_1 = 7.42\sigma_1$ ($p = 11$), $f_1 = 0.25$, $\vartheta^* = 2.5$, $\alpha_L = \alpha_U = 0.0025$
- Keeps the interval guarantee: Wald for $\theta\lambda$, bias-aware for β
- Chosen to minimize coefficient error over 70 synthetic cells; ridge off 35% sooner than the previous setting
- Synthetic, $\varepsilon = 1$: $\sigma_1 = 892$, $\sigma_2 = 575$, $n_{off} = 27.0k$
- Both keep the gate $\hat{\vartheta} \geq 1$; the previous setting ($f_1 = 0.1$, union bound) stays as a reference

Setting P (prediction)

- Exact radius at $\alpha_L = 0.75$ ($r_1 = 4.44\sigma_1$), $f_1 = 0.25$, $\vartheta^* = 1.5$, no intervals
- Tuned for test error only; same noise and same privacy guarantee as I
- Chosen over 140 cells, standardized and block-normalized, with the gate refusing at most 1% of releases
- Replaces P_0 ($\vartheta^* = 0.5$), whose gate refused most block-normalized releases on real data

From ε to σ and λ

Every input is public and fixed before any data are seen

1 δ

$\delta \le 1/(10N)$

N = people O protects. Smaller δ costs little (next slide)

2 Budget

tight conversion of (ε, δ)

$\sqrt{\rho_{\text{run}}} \gtrsim \sqrt{L+\varepsilon} - \sqrt{L}$

L = ln(1/δ). Two releases share ρ_{run}

3 Δ

$\Delta^2 = \frac{1}{2}(2O + R + Y)^2 + 2RY$

(R, O, Y) = squared bounds; Δ_G: B and C only

4 σ

$\sigma_1 = \Delta_G / \sqrt{2 f_1 \rho_{\text{run}}}$

$\sigma_2 = \Delta / \sqrt{2(1-f_1)\rho_{\text{run}}}$

f_1 = stage-1 share: 0.25 in I and P (previous: 0.1)

5 λ

$\lambda \approx \max\{0,\ 2\vartheta^\star \sigma_2 \sqrt{p} + r_1 - n\ell\}$

Planning value; $\ell = \lambda_{\min}(G)/n$ from reference data, else 0; $r_1 = q \cdot \sigma_1$

σ_2 / Δ δ \ ε	0.1	0.25	0.5	1	2	4	8
$\delta = 10^{-6}$	41.6	17.6	9.15	4.78	2.51	1.34	0.727
$\delta = 10^{-8}$	51.6	21.4	11	5.68	2.95	1.55	0.825
$\delta = 10^{-9}$	56	23.1	11.9	6.09	3.15	1.64	0.871

Planning formula

$$\sigma_2 \le \frac{\Delta}{\sqrt{2(1-f_1)}\left(\sqrt{L+\varepsilon} - \sqrt{L}\right)}$$

Simple zCDP bound: overstates σ_2 by 6–33%. Table at $f_1 = 0.1$; ×1.095 at $f_1 = 0.25$ (I, P).

From σ to the matched records

Two scaling laws, exact at first order: one pilot figure carries over to any ε

Prediction

$$n \propto \sigma \propto \Delta/\varepsilon$$

The ridge scales with σ, so the excess test error depends on n only through n / σ. Doubling ε roughly halves the records.

Coefficients

$$n \propto \sigma^2 \propto (\Delta/\varepsilon)^2$$

Once the ridge is off, DP error / sampling error ≈ $\sigma^2(1+\|\beta\|^2)\,\mathrm{tr}(\Sigma^{-2}) / (n\, s^2\, \mathrm{tr}(\Sigma^{-1}))$. Doubling ε cuts the records by about 3.5×.

Target	Setting	Measured at base ε	Transfer (× σ_2 or × σ_2^2)	Observed (v3.1)
Prediction within 10%	protein, I, ε = 2 (from ε = 1)	8.2k	4.2k	**4.1k**
Prediction within 10%	protein, I, ε = 4 (from ε = 1)	8.2k	2.2k	**2.1k**
Prediction within 10%	protein, P, ε = 2 (from ε = 1)	5.0k	2.6k	**2.5k**
Prediction within 10%	protein, P, ε = 8 (from ε = 1)	5.0k	715	**599**
Coefficients within 2× OLS	synthetic, I, ε = 4 (from ε = 2)	88k	24k	**27k**
Coefficients within 2× OLS	synthetic, I, ε = 8 (from ε = 2)	88k	6.8k	**8.0k**

Checked on v3.1 ($\delta = 10^{-9}$): prediction on protein with block normalization, coefficients on the synthetic population ($R^2 = 0.5$). Transfer = base count × the ratio of σ_2 (or σ_2^2) from Calibrate. The laws break down for ill-conditioned designs, at high R^2 where clipping bias dominates, and at small n.

What each choice costs

Reference: $\delta = 10^{-9}$, previous setting ($f_1 = 0.1$, $\vartheta^* = 2.5$), tight bounds, one run per budget

Change	σ_2	Records for prediction	Records for coefficients
Bounds 2× closer to the data (no extra clipping)	× 0.25	× 0.25	× 0.06
Split one unit row bound between the parties	× 0.64	× 0.64	× 0.41
Double ε	× 0.51–0.53	× 0.51–0.53	× 0.26–0.28
δ from 10^{-6} to 10^{-9}	× 1.20–1.32	× 1.20–1.32	× 1.44–1.73
Setting I instead of previous (exact r_1, $f_1 = 0.25$)	× 1.095	× 0.67	× 1.09–1.13
Setting P instead of I ($\vartheta^* = 1.5$, no intervals)	× 1.00	× 0.68–0.75	× 1.00
One budget shared by 5 runs (zCDP)	× 2.24	× 2.24	× 5.00
One budget shared by 5 runs (naive split of ε, δ)	× 4.81–4.97	× 4.81–4.97	× 23.10–24.70

Worked example: simulation population, $\delta = 10^{-9}$, settings I and P (same σ; $\Delta = 86.2$, $\Delta_G = 77.2$, $p = 11$)

ε	ρ_{run}	σ_1	σ_2	r_1 (I)	Ridge off from n: I	P	previous setting
0.5	0.0040	1737	1119	12,878	**53k**	31k	81k
1	0.0150	892	575	6,617	**27k**	16.2k	42k
2	0.0561	461	297	3,419	**13.9k**	8.4k	22k
4	0.2062	240	155	1,783	**7.3k**	4.4k	11.2k
8	0.7320	128	82	946	**3.9k**	2.3k	6.0k

6

Results

Accuracy in simulation and on real data

How we evaluated

Simulation population

Features on real-world scales (income, age, indicators, counts), correlated; 99.5% bounds. $R^2 \in \{0.3, 0.5, 0.9\}$; n from 100 to 100k; 200 cohorts per n; $\delta = 10^{-9}$.

Real data

29 UCI datasets from Wang's AdaSSP study; 10-fold CV. Each party normalizes its own block of every row, with bounds from a public reference split. $\delta = 10^{-6}$ against AdaSSP, 10^{-9} elsewhere.

Comparisons

Non-private least squares (the floor); predicting zero; AdaSSP on the joined table; full-record SSP, Ferrando–Wang–Sheldon intervals and TukeyEM.

What the experiments run

- A plaintext simulation of the exact v3.1 mechanism (two releases, ridge from stage 1, gate, bootstrap intervals), validated by 450 encrypted OpenFHE runs.
- Settings I and P, chosen on synthetic data by pre-registered rules; the previous setting ($f_1 = 0.1$) as a reference.
- Bias correction off. The earlier caveats (one release, a ridge from a known eigenvalue, the correction applied) no longer apply.

The full protocol runs on OpenFHE

All six phases on OpenFHE 1.5.1, R and O as separate processes with message transcripts; 80 tests

0

misaligned rows

PSI recovered exactly the true match set and alignment; A was exact in every encrypted run

$2^{17.8–18.8}$

measured encryption error

7.8–8.7 bits below the proven worst case $B_{tot} = 2^{26.6}$ that sets the scale

22–43 s

per run, one core

1k–64k register rows; O's CPU about 260 s at 1M rows (76 s on 4 cores), extrapolated

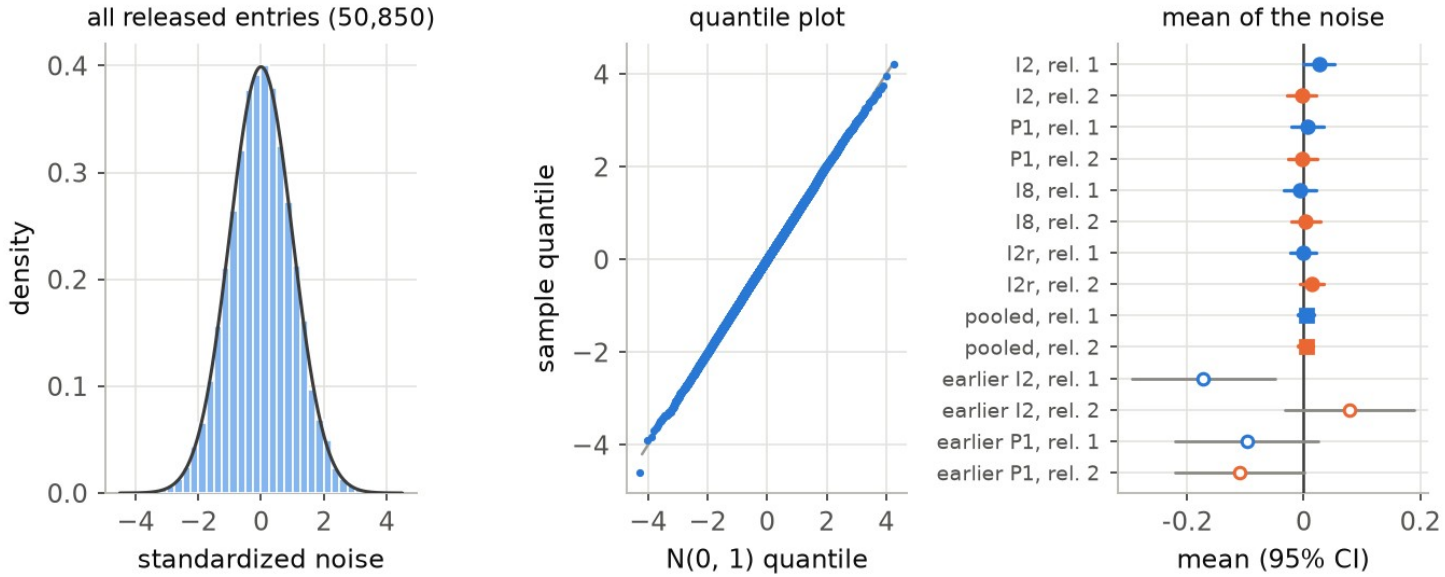
Cost per run, measured at $\bar{n}_O = 65{,}536$ (one core)

Measured to 64k rows

Step	Communication	O's computation	Rounds
Link	n_R^{max} OPRF calls; PSI (27 MB)	Preprocessing linear in $\bar{n}_O$; per-query evaluation (15.3 s)	2–3
Upload	$(d_R + 2)\cdot m$ half-size ciphertexts (24 MB)	$(d_R + 2)\cdot m$ key conversions (0.3 s)	1
Aggregate	None	$\lvert\mathcal{E}\rvert\cdot m$ products; $\lvert\mathcal{E}\rvert\cdot \log_2(N/2)$ rotations (19.8 s; 5.4 s on 4 threads)	0
Release (×2)	Two ring elements each (1 MB in all)	Re-randomize, partial decryption, N exact samples (0.6 s)	1 each

$m = \lceil \bar{n}_O/(N/2) \rceil$ chunks; $\lvert\mathcal{E}\rvert$ = released entries. Key generation adds 20 MB. Traffic 73 MB at 65,536 rows; 88–422 MB extrapolated to 100k–1M.

Faithful under encryption



What was run

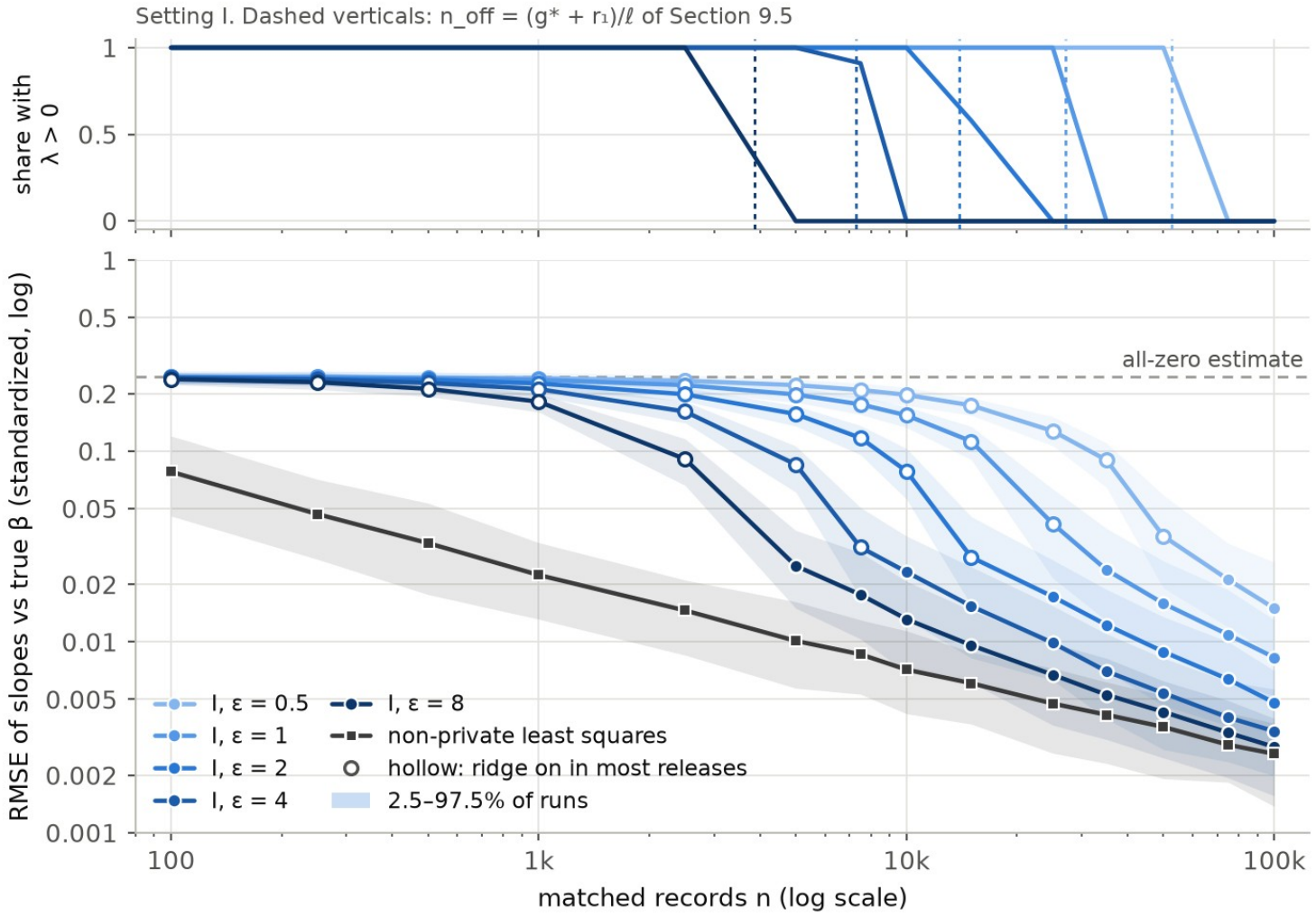
- 450 OpenFHE runs, n = 10,000 matched in a 16,384-row register: I at ε = 2 (250), P at ε = 1 (100), I at ε = 8 (100)
- No release refused; A exact; λ and estimates rebuilt from the recorded noise to 10^{-11}
- λ in the simulated 95% range in 95/100, 144/150, 92/100, 100/100 runs

What it shows

- The decrypted noise has its calibrated law: mean +0.006 and +0.004 (s.e. 0.007, 0.006), s.d. 0.995 and 1.006, KS p = 0.13 and 0.48
- Wald coverage 0.954 with the ridge on and 0.958 with it off: encryption changes only the noise draw

Accuracy against intersection size

Simulation, $R^2 = 0.5$, setting I, $\delta = 10^{-9}$, 200 cohorts per n

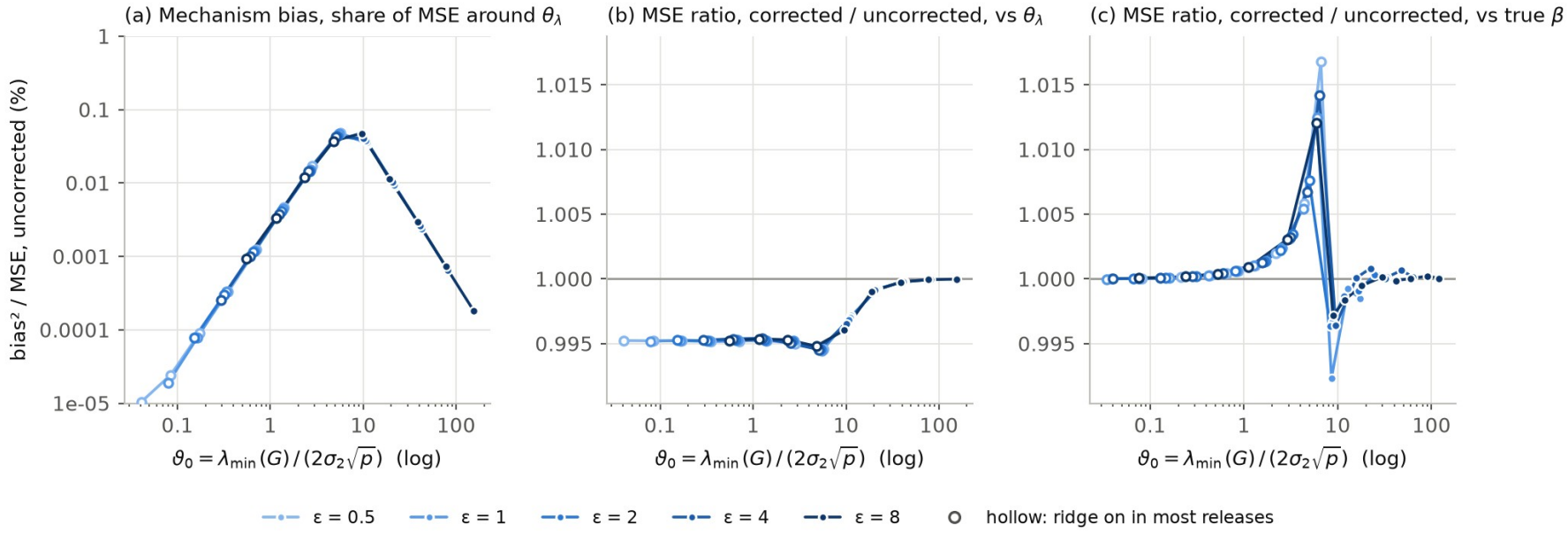


Within 2× of non-private least squares at

ε = 8	**n ≈ 8.0k**
ε = 4	**n ≈ 27k**
ε = 2	**n ≈ 88k**
ε = 1	**n > 100k**

- Small n: the ridge pulls the estimate towards 0
- Ridge off at n_{off} = 3.9k–53k, 35% sooner than before
- Prediction within 10% much earlier: 20k (I), 13.7k (P) at ε = 1

Where the error comes from when σ is large



Error anatomy

- $\vartheta_0 = \lambda_{\min}(G) / (2\sigma_2\sqrt{p})$
- Mechanism bias under I: ≤ 0.19% of the MSE around θ_λ (cell average; 0.24% worst case)
- The optional correction gains only 1.4–1.7%: it stays off
- At large n, clipping shifts the target slightly (any DP method pays this)

So what?

- The ridge, not the noise, limits accuracy at small n: tighter bounds and more matched records help most
- ϑ_0 compares the weakest part of the signal with the size of the noise, and it grows like n/σ, roughly n·ε. Figure: v3.1 at the previous setting.

Intervals that hold up

Setting I, $\delta = 10^{-9}$: simulation of the exact mechanism, real data with calibrated noise, and encrypted runs

Check	Size	Ridge	Wald interval for θ_λ (target 0.955)	Bias-aware interval for β (target ≥ 0.95)
Synthetic population	70 cells × 200 releases	on in 43 cells, off in 27	0.945–0.966 in every cell	0.930–0.964 with the ridge off; unbounded while it is on
Repeats, previous setting	10,000 releases each	off (synthetic); on (protein)	ε = 8: 0.9554; protein: 0.9548–0.9577	—
Real data, calibrated noise	48 cells × 1,000 releases	on in 40 cells, off in 8	0.945–0.964; protein 0.952–0.963	not checkable on a fixed cohort
Encrypted runs (OpenFHE)	250 and 100 runs	on (ε = 2) / off (ε = 8)	0.954 / 0.958	—

Reading the table

- Pooled over coefficients and releases; target 0.955
- Wald hits its target with or without the ridge, plaintext or encrypted
- The β interval is valid, informative only once the ridge is off

For contrast

A naive Wald interval read as an interval for β covered only 9–85% while the ridge was on (41 cells). Ferrando–Wang–Sheldon intervals are finite there, but cover θ_λ only 0.57–0.86. Naming the target matters.

Real data against central DP

Test MSE ÷ own non-private MSE at $\varepsilon = 1$, $\delta = 10^{-6}$, 10-fold cross-validation; v3.1 with block normalization

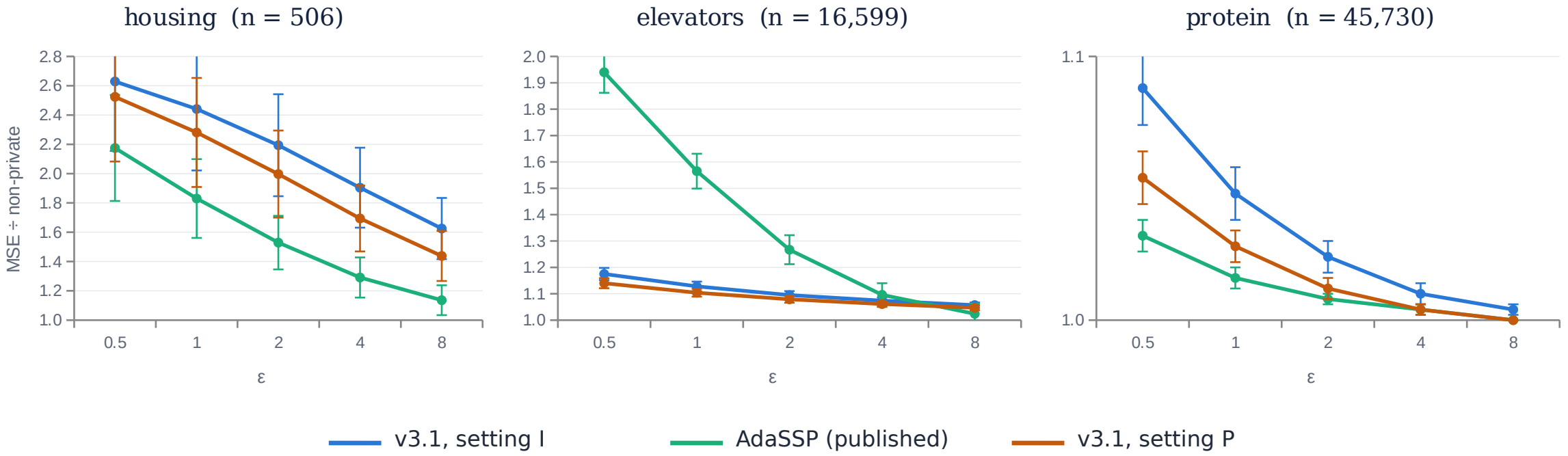
Dataset	n	d	Predict zero	AdaSSP (published)	v3.1 I	v3.1 P
housing	506	13	2.81×	**1.83×**	2.44×	2.28×
wine	1,599	11	2.83×	1.75×	1.87×	**1.73×**
airfoil	1,503	5	1.94×	**1.10×**	1.43×	1.27×
concrete	1,030	8	2.87×	**1.53×**	2.56×	2.26×
bike	17,379	17	3.85×	**1.04×**	1.38×	1.22×
elevators	16,599	18	4.21×	1.56×	1.13×	**1.10×**
pol	15,000	26	2.56×	**1.04×**	1.53×	1.38×
kin40k	40,000	8	1.00×	1.00×	1.00×	1.00×
protein	45,730	9	1.40×	**1.01×**	1.02×	**1.01×**

1.0× = as good as each method's own non-private fit; lower is better; bold = lowest private ratio.

AdaSSP needs the joined table and protects the whole record, but its guarantee is only nominally stronger: under our replace-one adjacency its published constants give ε up to 28% above nominal. Elevators: block normalization's own non-private fit is 2.6× worse, so AdaSSP still wins in absolute MSE. Predict zero: ÷ Wang's non-private fit.

Across privacy levels

Test MSE ÷ own non-private MSE, three datasets of increasing size, δ = 10⁻⁶; bars: 95% CI over folds



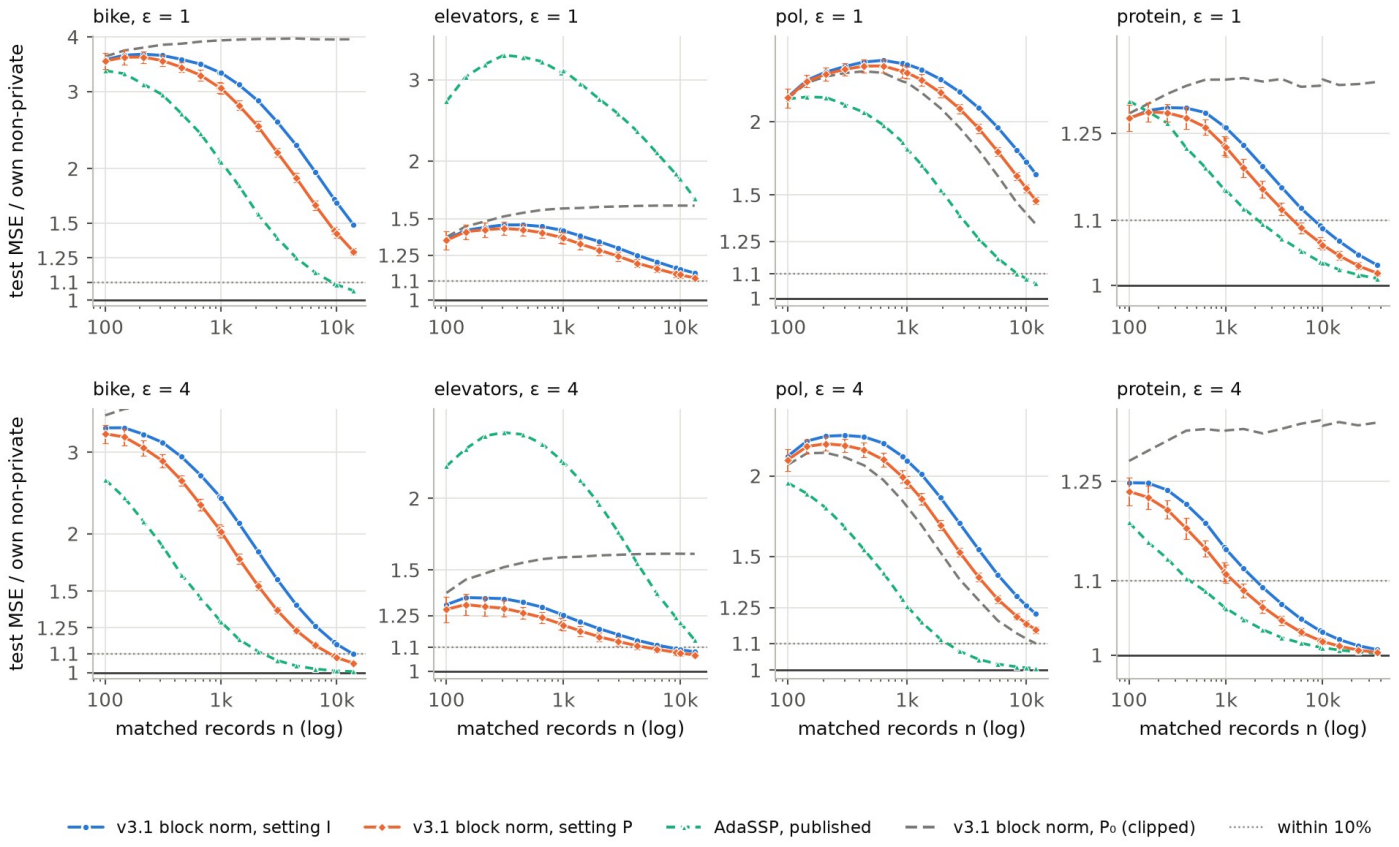
29 datasets, vs AdaSSP	ε = 0.5	ε = 1	ε = 8
P: better / worse	6 / 19	5 / 20	3 / 15
I: better / worse	6 / 19	5 / 20	1 / 21
Median gap (%), P / I	+10 / +15	+14 / +25	+4 / +11

Better / worse: paired over folds, 95% CI, absolute test MSE; the rest are ties.

AdaSSP on the joined table is ahead on most datasets; the gap shrinks with ε. The cause is the stage-1 ridge, 1.8× (P) and 2.8× (I) AdaSSP's under the same guarantee, sized for valid intervals; v3.1's stage-2 noise is lower.

How many matched records on real data?

Test MSE ÷ own non-private MSE at the same n; nested training subsets; block normalization; δ = 10⁻⁹



Matched records to come within 10% of own non-private

v3.1 P / v3.1 I / (AdaSSP, published)

	ε = 1	ε = 2	ε = 4	ε = 8
bike	**>13.9k** >13.9k (9.1k)	**>13.9k** >13.9k (4.5k)	**8.6k** 13.6k (2.3k)	**4.5k** 7.3k (1.2k)
elevators	**>13.3k** >13.3k (>13.3k)	**9.5k** >13.3k (>13.3k)	**5.1k** 8.0k (>13.3k)	**2.5k** 4.1k (8.8k)
pol	**>12.0k** >12.0k (8.5k)	**>12.0k** >12.0k (4.3k)	**>12.0k** >12.0k (2.2k)	**10.5k** >12.0k (1.0k)
protein	**5.0k** 8.2k (2.2k)	**2.5k** 4.1k (992)	**1.2k** 2.1k (427)	**599** 985 (166)

- Doubling ε roughly halves the records (protein, P: 5.0k → 2.5k → 1.2k → 599)
- On protein P needs 2.3–3.6× and I 3.7–5.9× the records of AdaSSP
- Elevators reaches 10% of its own (worse) non-private fit before AdaSSP

Other baselines

Full-record SSP

Noise on R’s block A too, the protocol otherwise unchanged: test MSE about 1% higher on the 29 datasets, 7–18% more records on the synthetic population, about 1.16× the run time.

FWS intervals

Ferrando–Wang–Sheldon bootstrap on our stage-2 release: covers θ_λ only 0.57–0.86 while the ridge is on. For β it equals our interval once the ridge is off, and stays finite where ours is unbounded.

TukeyEM

Least squares on subsets, chosen by the exponential mechanism; needs no bounds. It releases nothing on 25 of 29 datasets at $\varepsilon \le 2$, and is worse than v3.1 (P) on 24–25.

What they tell us

- Keeping A exact is a modest accuracy gain, and spends no budget or circuit work on statistics R already holds.
- Dropping the bounds does not help at these sizes.
- For β while the ridge is on, a bootstrap without the ridge helps from $\vartheta_0 \approx 1$, about a quarter of n_{off}; below that only θ_λ has a useful interval.

Limits and open items

By design

- Honest-but-curious parties only; a deviating researcher needs zero-knowledge input proofs
- R learns which of its cohort are in the register
- Exact-identifier linkage; linear models; model-based variance
- O's work grows with the padded register, so the scope Φ matters

Still to do

- Runs at 100k–1M register rows; the PSI item length needs about 59 bits at 1M
- Redesign the stage-1 rule so the ridge does not overshoot: it is the gap to AdaSSP
- Open proofs: the fixed-run admission rule under adaptive requests; β intervals when clipping makes errors heteroskedastic; the PSI noise bound for OpenFHE’s RNS BFV; its error sampler, checked only numerically
- Author decisions still pending (e.g. a noise-ratio-dependent P, publishing precision)

Summary

1. A researcher and an agency can fit a joint regression without pooling: linkage by labeled PSI, sums under split-key encryption, DP noise added inside O's decryption share

2. O’s view reveals only public sizes; R gets (ε, δ)-DP statistics, composed across runs by O’s ledger. Proven for honest-but-curious parties and implemented end to end on OpenFHE

3. Valid intervals for the ridge target (0.945–0.966 in simulation). Protein comes within 10% of non-private at 5.0k (P) and 8.2k (I) matched records for ε = 1

4. Trails central DP on the joined table by a median 14% (P) / 25% (I) at ε = 1, because its ridge is sized for valid intervals